Bomare Assembly Line Optimizer

Technical Report

1. Purpose and Scope

A manufacturing line consists of many tasks, such as assembling parts, installing components, and testing products. Some tasks depend on others, so the factory needs to decide how to organize the line, group tasks into stations, and assign workers.

Our tool takes **task data, raw-material stocks, and business information** such as shift length, product price, worker salary, and WIP holding cost. It then proposes:

- **Reduce the number of workers and labor cost**
- **Reduce WIP** (unfinished products waiting between stations)
- **Increase production during a shift**

The goal is to find a good balance between these objectives and avoid bottlenecks.

2. System Overview

The project has three layers. The **interface layer** receives the user's input. The **bridge layer** cleans the input and formats the output. The **core layer** does the real work: it checks the line, builds starting layouts, simulates a shift, and searches for the best layout.

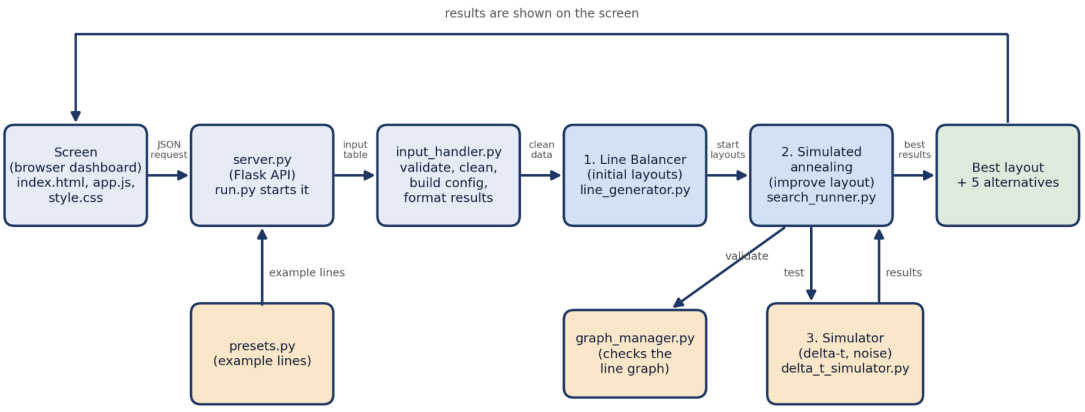


Figure 1 – Architecture and call chain

3. Project Structure

```
test_bomare/
├── run.py                      start the web server
├── server.py                   Flask routes (the API)
├── input_handler.py            validation, config building, result formatting
├── presets.py                  ready-made example line
├── requirements.txt            Python dependencies
├── engine/
│   ├── graph_manager.py        CORE – line graph and its rules
│   └── delta_t_simulator.py    CORE – shift simulation
├── optimization/
│   ├── line_generator.py       CORE – starting layouts (line balancing)
│   └── search_runner.py        CORE – simulated annealing search
└── static/
    ├── index.html              dashboard page
    ├── app.js                  dashboard logic
    └── style.css               dashboard styling
```

3.1 What each file does

File	Role	Works with
`run.py`	Entry point. Starts the Flask server on port 5000 (`python run.py`).	`server.py`
`server.py`	Defines the web routes: dashboard page, presets, validate, optimize, CSV import/export, health check.	`input_handler.py`, `presets.py`
`input_handler.py`	The only bridge between dashboard and optimizer. Cleans rows, validates the graph, converts hours to seconds, builds `SearchConfig`, calls the search, and reshapes the result for the dashboard.	`server.py`, `search_runner.py`
`static/*`	Browser dashboard: editable task table, parameter form, results view, charts.	`server.py` (JSON)
`graph_manager.py`	Builds the task graph from the table and rejects invalid structures.	`search_runner.py`
`delta_t_simulator.py`	Simulates one shift of the line and returns output, WIP, and station metrics.	`search_runner.py`
`line_generator.py`	Computes pacing targets and builds several valid starting layouts.	`search_runner.py`
`search_runner.py`	Runs the optimization: seeds, moves, simulation, fitness, accept/reject.	all three core files, `input_handler.py`

3.2 API routes (server.py)

Route	Method	Purpose
/	GET	Serves the dashboard page.
/api/templates	GET	Returns an example table .
/api/validate	POST	Checks a table and returns errors, warnings, and a short summary.
/api/optimize	POST	Runs the full pipeline and returns the best layout and alternatives.
/api/export-csv, /api/import-csv	POST	Download or upload the table as a CSV file.
/health	GET	Simple health check.

3.3 How to run

```
pip install -r requirements.txt
python run.py          # then open http://localhost:5000
```

4. Input Data Model

Everything starts from one table. Each row is either a **Task** (work to be done) or a **Resource** (a raw-material stock).

Column	Meaning
Id	Unique name of the row (for example T1, R1).
Type	Task or Resource.
Relation	List of predecessor ids: the rows that must supply this row before it can run.
CycleTime	Duration of the task in seconds.
Quantity	How many units of this row are consumed by the row that uses it (by its successor).
Available	Starting stock (resources only).
operation_type	manual (more workers shorten the time) or machine (fixed time, cannot be sped up).
min_operators, max_operators	Smallest and largest crew allowed for the task. Machines always use the minimum.

Business parameters entered with the table: shift duration (hours), unit price, worker salary, WIP holding cost, and two **optional** limits: **available workers** and **target quantity**.

5. The Four Core Files

5.1 engine/graph_manager.py – the line graph

Purpose. It turns the table into a directed graph (an arrow from each predecessor to the row that needs it) and makes sure the line is physically sensible.

Main class: `RawTableGraph`

Method	What it does
`build_dag(df)`	Adds one node per row and one arrow per predecessor.
`get_topological_order()`	returns a list of all the tasks in a valid order, so every task comes after all the tasks it depends on.
`is_valid_station_group(ids)`	Returns true if the given tasks form one connected piece of the graph. Used to make sure a station never contains unrelated tasks.

Two rules are enforced when the graph is built:

- **No loops.** A circular dependency raises an error.
- **No branching.** A row may feed only one other row (out-degree of at most 1). The line therefore looks like a tree whose branches converge into a single final task.

5.2 optimization/line_generator.py – starting layouts

Purpose. It builds good, valid starting layouts using classic line-balancing logic. These are later improved by the search. The file works in three phases.

Phase 1 – PacingCalculator: how fast must a station be?

Value	Definition
Takt time	`shift_duration / target_units`. The time available per unit to reach the target.
CT_ideal	`total cycle time/ available workers`. The cycle time if work were shared perfectly.
CT_tentative	The smaller of takt time and CT_ideal.
CT_floor	the time of the slowest task that cannot be sped up (a machine task, or a manual task at its maximum workers). No station can be faster than this.
CT_target	`max(CT_tentative, CT_floor)`. The cycle-time limit used to fill stations.

The user can give a production target ,a number of workers, or both. If there is no target or no shift duration, the tool uses worker-only mode: takt time is set to 0, so the pacing depends only on CT_ideal. If there are no workers, CT_ideal cannot be computed, so the pacing depends only on the takt time.

To find CT_floor, the tool does not check every task. It puts all tasks in a max-heap, so the slowest task comes out first. For each task it takes out, it first compares the task's cycle time with CT_tentative. If the time is not above CT_tentative, the search stops for both machine and manual tasks, because all the remaining tasks are faster too. Otherwise:

- **Machine task:** its time cannot be reduced, so it is taken directly as CT_floor and the search stops.
- **Manual task:** its best possible time is its cycle time divided by its maximum number of workers. If this is still above CT_tentative, so it is taken directly as CT_floor and the search stops otherwise search continues with the next task.

Phase 2 – LineGenerator: filling stations

1. Start with the tasks that have no unfinished predecessors (the "free pool").
2. Open a new station and pick one free task at random as its first task.
3. Decide the task's crew (the **P-factor**): machines keep their fixed crew; manual tasks start at the minimum crew and add workers, one at a time, until the task time is at most CT_target or the maximum crew is reached.
4. Try to add the **next connected task** (a successor of the last one added). It is accepted only if the station time stays at or below CT_target.
5. When no connected task fits, close the station and start a new one. Repeat until all tasks are placed.

Phase 3 – LineEvaluator: checking a layout

- **Station time** = sum of the effective task times (time divided by crew for manual tasks).
- **Bottleneck** = the slowest station time.
- **Station crew** = the largest crew needed by any one of its tasks (workers rotate inside a station).
- **Total workers** = sum of station crews, compared with the available workers to raise an overflow flag.

LineBalancer – producing several candidates

`LineBalancer.run()` computes the pacing once, then repeats Phases 2 and 3 with different random seeds. Duplicate layouts are discarded. It stops at 5 distinct candidates, or after 100 attempts if the line is too rigid to give 5.

5.3 engine/delta_t_simulator.py – the shift simulator

Purpose. It estimates how a given layout really behaves during one shift: how many units come out, how much work piles up, and how busy each station is. It is a **next-event (delta-t) simulation**: instead of moving time forward in tiny fixed steps, it jumps straight to the next moment when something finishes.

Input: the station table

group	Relation	Type	Quantity	CycleTime	P_factor	Available
Row ID	Parent rows	Resource / Task	Units consumed by successor row	Cycle time	Crew size	Current stock / output

At the start the table is converted into small NumPy arrays so the main loop runs quickly without pandas.

Readiness rule

```
A station is READY  if  for every parent d:  Available[d] >= Quantity[d]
```

A station that is not ready is **frozen**: its timer does not run, and it counts as starved.

The event loop

1. Give every station a first timer: cycle time plus a random transition time, with random variation (see below).
2. Find all ready stations. If none is ready, the line is stalled: credit the remaining time and stop.
3. Compute `dt` = the smallest timer among ready stations.
4. If `dt` goes beyond the end of the shift, record the partial interval and stop.
5. Otherwise advance the clock by `dt`. Add `WIP × dt` to the time-weighted WIP total. Add `dt` to the busy time of ready stations and to the starved time of frozen ones. Reduce every ready timer by `dt`.
6. For each station whose timer reached zero: remove `Quantity` from each parent, add 1 unit to its own `Available`, count the units produced by that station , and draw a new timer.
7. Go back to step 2.

Randomness (those are assumptions that can be modified)

- **Transition time:** a random value between 0.5 and 2.0 minutes is added to each cycle.
- **Variation:** the time is multiplied by a lognormal factor with sigma = 0.05. The factor is built to keep the **average unchanged**, so it adds realism without making the line slower on average.

$T_{variable} = cycle_time \times X \qquad X \sim Lognormal(\mu = -\sigma^2/2, \sigma^2) \qquad \text{where} \qquad E[X] = 1$
Therefore: $E[T_{variable}] = cycle_time \times E[X] = cycle_time$

- Each simulation uses its own seeded random stream, so results can be repeated exactly.

Output metrics

Metric	How it is computed
`throughput_units`	Units stored at the final (sink) station.
`avg_wip`	Time-weighted average of unfinished work: stock waiting at stations (the final products are excluded) plus the number of ready stations.
`total_workers`	Sum of the crews of all stations.
`line_efficiency_pct`	Total busy time divided by (simulated time × number of stations).
`station_metrics`	Busy %, starved %, and units produced for each station.
`event_cap_reached`	Safety flag. The loop stops after 2,000,000 events and reports it instead of failing silently.

5.4 optimization/search_runner.py – the optimizer

Purpose. It searches for the best layout using **simulated annealing**. It is the controller of the whole project: it calls the other three core files and returns the final answer.

Step A – setup(): from table to Problem

The raw table is cleaned (column names, ids, relation lists, numbers) and checked: no duplicate ids, no unknown predecessors, resources must be sources, exactly one final task. `RawTableGraph` then verifies no loops and no branching. The result is a `Problem` object that stores the lookups used everywhere else (task info, predecessors, successor, resource consumer, topological order).

Step B – Layout: the thing being optimized

A **Layout** is only two things: a partition of the tasks into stations, and a crew size for each station. Everything else (station time, graph between stations, simulator table) is always **recomputed from these two things**, never edited by hand. Each station must follow three rules:

- its tasks form one **connected** group,
- it has exactly **one exit task** (a task whose successor lies outside the station),
- the graph between stations stays **free of loops**.

Station time is computed by `refresh()`: each task's duration multiplied by how many times it runs per station output, where manual tasks use `cycle_time / min(crew, max_operators)` and machines use their fixed time. The crew is kept inside the allowed bounds for the station's tasks.

Step C – make_seeds(): starting layouts

The tasks are passed to `LineBalancer` (from `line_generator.py`), which returns up to 5 candidate layouts. Each is converted to a `Layout` and fully validated. Invalid ones are discarded with a warning.

Step D – the four moves

To explore alternatives, the search changes the current layout with one of four moves. Only legal moves are listed.

Move	Effect
`crew`	Add or remove one worker in a station, within its allowed range. • minimum crew = the largest minimum-operator requirement across the tasks in the station • maximum crew = the largest manual-task operator cap in the station, if any; otherwise the minimum crew
`split`	cuts a station at a random chosen task when that task has at most one predecessor inside the station, creating two valid sub-stations
`exit_fwd`	Move the station's exit task into the next station downstream, if the station graph stays loop-free.
`merge`	Join a station with the station that consumes its output. The merged crew is the larger of the two.

And after each move we recalculate their crew size , times , and simulation quantities and their dependencies

Step E – compile_for_simulator()

A layout is converted into the table that the simulator understands. Resources come first. Then each station becomes one **Task row** whose parents are the resources it uses and the upstream stations that feed it. Its `CycleTime` is the station time and its `P_factor` is the crew. `Quantity` is scaled by the number of times the consuming task runs per station cycle, so that material use matches the time that is charged.

Step F – evaluation and fitness

The `Evaluator` simulates each layout **3 times** (the same random seeds for every layout, so layouts are compared fairly) and averages units and WIP. Results are **cached** by layout signature, so a layout is never simulated twice. The score is a profit-style value:

```
fitness =  unit_price × units
         − unit_price × max(0, target − units)        (only if a target is set)
         − wip_holding_cost × avg_wip
         − worker_salary × min(available_workers, workers)
         − 1.5 × worker_salary × max(0, workers − available_workers)

If no worker limit is given:  − worker_salary × workers
```

A status flag is also returned: **Red** = target not met, **Yellow** = target met but the worker budget is exceeded, **Green** = everything satisfied (the dashboard shows "OPTIMAL").

Step G – the annealing loop

1. Simulate all seeds and start from the best one.
2. Pick one random legal move and build the neighbour layout.
3. Evaluate it. Let `delta = new fitness − current fitness`.
4. If `delta > 0`, accept. Otherwise accept with probability `exp(delta / temperature)`. This lets the search escape poor local solutions early on.
5. Keep the best layout seen so far. Multiply the temperature by the cooling rate.
6. Stop after the maximum iterations, or after a long run with no improvement.

Setting (fixed in `input_handler.py`)	Value
Starting layouts	5
Iterations (maximum)	200
Start temperature / cooling rate	100 / 0.98 per iteration
Simulation runs per layout	3
Stop if no improvement for	100 iterations
Master seed	42 (results are repeatable)

Final result: the best layout, the top 5 other layouts found (alternatives), the history of best fitness, the number of iterations, whether it stopped early, and how many different layouts were evaluated.

6. End-to-End Flow

This is the journey of one request, from the click on the dashboard to the result on screen.

1. **Input.** The user loads a preset or edits the table, sets the shift length, price, salary and WIP cost, and clicks Run.
2. **Check.** The server cleans the table and makes sure it is a valid assembly line: every predecessor exists, there are no loops, no branching, and only one final task. If something is wrong, the user gets a clear error message and nothing runs.
3. **Prepare.** The parameters are checked and converted (hours become seconds), and the table is turned into the model the optimizer works on.
4. **Starting point.** The program calculates the target time per station and builds up to 5 different starting layouts. The best one becomes the first candidate.
5. **Improve.** The optimizer repeats a simple loop:
    - It changes the layout slightly (add or remove a worker, split a station, move a task forward, or merge stations).
    - It simulates a full shift 3 times and measures units produced and WIP.
    - It scores the result: revenue, minus the missed-target penalty, WIP cost, and salaries.
    - It keeps the change if it is better, or sometimes even if it is slightly worse, to avoid getting stuck.
6. **Answer.** The best layout and the top alternatives are formatted (stations in order, workers per task, status flag) and sent back to the browser.
7. **Display.** The dashboard shows the station layout, the key metrics, the alternative layouts, and the chart of how the score improved during the search.

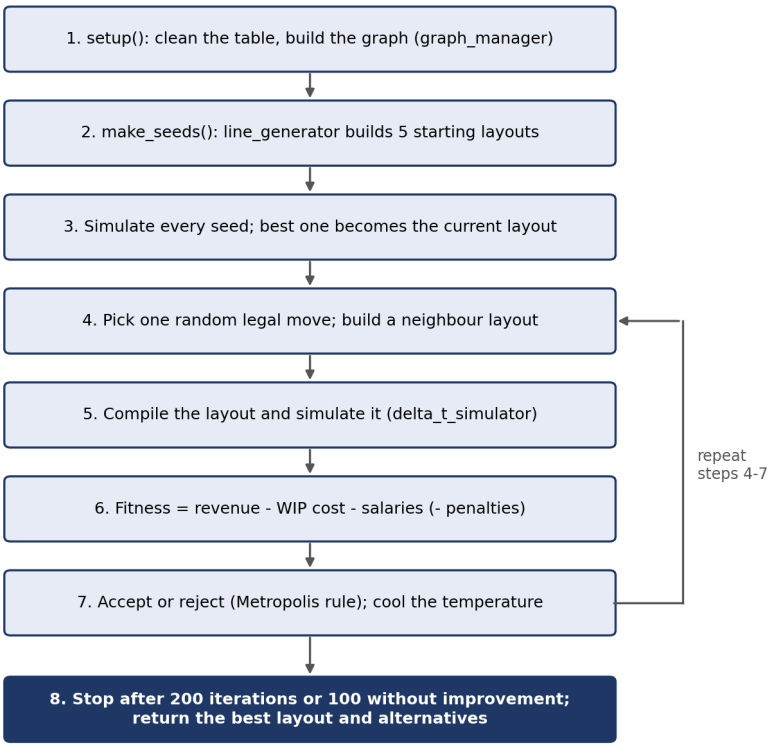


Figure 2 – The optimization loop inside search_runner.py

8. Assumptions

- **Resources** are pure sources (no predecessors), start with a given stock, and are consumed only by one task.
- **Workers rotate inside a station,** so a station needs as many workers as its most demanding task.
- **Manual time scales with crew:** time = cycle time / workers, up to the maximum crew. Machine time is fixed.
- **All times are in seconds** (cycle times and shift length; the dashboard shift is entered in hours and converted).
- **Buffers between stations are unlimited,** so a station is only blocked by missing input, never by a full output.
- **No breakdowns, breaks, absences, or setup changes** other than the small random transition time and variation.
- **Random variation** has a mean of 1, so it does not change average speed.

9. Methods Summary

Problem	Method	File
Check the line structure	Directed acyclic graph, cycle check, out-degree rule	`graph_manager.py`
Decide the speed each station must reach	Takt time, ideal and floor cycle time	`line_generator.py`
Build starting layouts	Successor-first greedy packing with random restarts	`line_generator.py`
Estimate real output and WIP	Next-event stochastic simulation (lognormal noise)	`delta_t_simulator.py`
Improve the layout	Simulated annealing with four neighbourhood moves	`search_runner.py`
Compare layouts	Profit-style fitness with penalties; common random numbers; caching	`search_runner.py`

10. What We Use to Build This Project

Programming languages

Language	What we use it for
Python 3	The whole backend: graph checks, line balancer, simulator and optimization.
JavaScript	The dashboard in the browser: tables, graphs and charts.
HTML and CSS	The structure and the style of the dashboard page.

Python libraries

Library	What we use it for
Flask	Runs the web server and the API that the dashboard calls.
NetworkX	Builds the precedence graph, finds loops , `Unidirectional flow`
pandas	Reads and cleans the input table and the CSV files.
NumPy	Fast math and random numbers, such as the lognormal noise in the simulation.